

•



- [Architecture](#)
- Pulse Talks to Case Manager

On this page

Pulse Talks to Case Manager

Case Manager works with the events that it receives from Pulse. Through different components, those events arrive in Case Manager where analysts can take action on them. In this section, the connection between Pulse and Case Manager is explained, giving you an overview on how events are sent from Pulse to Case Manager.

Pulse Talks to Case Manager📄

In this section, we will talk about how Case Manager receives the information from Pulse. This fault-tolerant communication decouples the Case Manager data layer from Pulse (through the Alert Storage Service and RabbitMQ). This connection allows Case Manager to fail and recover without message losses and distribute the workload across all stateless Case Manager instances to achieve a horizontal, easy-to-scale architecture.

You can see where Pulse-Case Manager connection stays in the general architecture in the following diagram:

The Alert Storage Service send messages to a RabbitMQ queue. Case Manager subsequently retrieves messages from that very same queue and stores these messages into its datastore. This connection is bridged by an API so that Case Manager can change its datastore without affecting external systems.

In this model, we can see that Pulse's RabbitMQ cluster is used, connecting to it through Pulse's messaging-lib (which relies on AMQP). A virtual host is created with an **Events** queue inside, this queue is persistent and dimensioned to handle downtime and it contains heterogeneous messages.

Alert Storage Service

The **Alert Storage Service** is a Workflow Custom Service used by Case Manager in order to populate its database with the events that are being processed by Pulse. More specifically, the Alert Storage Service listens to the events that arrive at the end of a Pulse data processing workflow in order to store these events into Case Manager's database.

Data Workflow

In order to listen to the Pulse workflow's dead-letter, explanations and event status listeners, the Alert Storage Service connects to RabbitMQ through the messaging-lib API. RabbitMQ distributes messages fairly to the multiple Case Manager instances subscribing from the "Events" queue. When messages are received by Case Manager, they are identified by their `@type` attribute and stored into the Case Manager datastore.

As for dequeuing messages from the events queue and storing data into Case Manager's datastore, both are atomic operations. In order for this to happen, AMQP transactions (encapsuled by messaging-lib) and database transactions are used.



Messaging Dependence

Some messages depend on others:

- **Explanation** messages depend on **Event** messages.
- **Status change** messages depend on **Event** messages.

If there are multiple Case Manager instances, messages can be concurrently dequeued and processed out of order: the **status change** and **explanations** messages will be processed as an error (as for an FK constraint violation), returned to queue and redelivered. If multiple messages are returned to their original queue, they will always keep their original order and thus be queued in the same position.

Logging and Recovery📖

A custom logger holds all communications between the Alert Storage Service and Case Manager in order to log enqueues, dequeues and processing errors. Recovery from processing errors is manual. The operations team will be notified to review issues by a system monitoring of the log for errors.

Learn More

Case Manager also helps improving Pulse's detection models. Know more about this in [Pulse Gets Feedback from Case Manager](#).